

dataprep_joining_datasets

RR

Quarto

```
library(sf)
library(terra)
library(dplyr)
library(exactextractr)
library(foreign)
library(readr)
library(sf)
library(dplyr)
library(purrr)

# List all folders starting with LIST_PARCELS
municipality_folders <- list.dirs("000_CADASTRAL_FIXXX",
                                recursive = FALSE, full.names = TRUE)

# For each folder, construct the path to the .shp file
shapefile_paths <- file.path(
  municipality_folders,
  paste0(basename(municipality_folders), ".shp"))

# Read all shapefiles into a named list
parcels_list <- shapefile_paths %>%
  set_names(basename(municipality_folders)) %>%
  map(~ st_read(.x, quiet = TRUE))

# Combine and filter all municipalities into one single sf object
parcels_all <- bind_rows(parcels_list, .id = "Municipality") %>%
  filter(PID != "0", TENURE_TY != "Crown Land")
```

```
# 6. Rename Municipality to numeric codes (alphabetical order)
parcels_all <- parcels_all %>%
mutate(Municipality = factor(Municipality,
                             levels = sort(unique(Municipality))),
       Municipality = as.integer(Municipality))

# 7. Drop irrelevant columns
parcels_all <- parcels_all %>%
  select(-VOLUME, -FEAT_NAME, -STRATA_LEV, -COMP_AREA, -MEAS_AREA, -UFI, -FMP,
        -CREATED_ON, -SHAPE_AREA, -SHAPE_LEN)

# 8. Retain one PID from any PID Dupes
parcels_all <- parcels_all %>%
  distinct(PID, .keep_all = TRUE)
```

```
library(stringr)

# Read address points data into R
address_points <- st_read("LIST_ADDRESS_POINTS_STATEWIDE/list_address_points_statewide.shp",
                          quiet = TRUE)

# Remove irrelevant Variables
address_points <- address_points %>%
  select(PID, ST_NO_FROM, STREET, ST_TYPE, LOCALITY, STATE, POSTCODE, geometry)

# Join them into a full address
address_points <- address_points %>%
  mutate(
    FULL_ADDRESS = str_c(
      ST_NO_FROM, STREET, ST_TYPE, LOCALITY, STATE, POSTCODE,
      sep = " ") %>%
    str_squish()) %>%
  select(-ST_NO_FROM, -STREET, -ST_TYPE, -LOCALITY, -STATE, -POSTCODE)
```

```
# Join parcels_all and address_points

joined_data <- parcels_all %>%
  inner_join(st_drop_geometry(address_points), by = "PID")
```

```
# --- 0) Paths (edit as needed) -----
esm_dir <- "ESM_Tiff/ESM_Tiff" # folder containing tif + tif.vat.dbf
```

```

barley_tif <- file.path(esm_dir, "Barley.tif")
barley_vat <- file.path(esm_dir, "Barley.tif.vat.dbf")

# --- 1) Load raster + VAT -----
r <- rast(barley_tif)

vat <- read.dbf(barley_vat, as.is = TRUE)

# Heuristics: find the value code column and a human-readable label column
val_col <- grep("^value$", names(vat), ignore.case = TRUE, value = TRUE)
if (length(val_col) != 1L) stop("Could not identify the VAT 'Value' column.")
# Prefer columns that look like suitability/class labels
lab_col <- grep("(suitab|class|label|cat)", names(vat), ignore.case = TRUE, value = TRUE)
if (length(lab_col) == 0L) {
  # fallback: if no label-like column, just use the value as character
  lab_col <- val_col
}

# Attach labels to the raster as a categorical RAT for plotting/inspection
levels(r) <- data.frame(ID = vat[[val_col]], Category = vat[[lab_col]])

# Locate the address point (for Prototype Testing Purpose)
target_addr <- "401 BRESNEHANS ROAD LITTLE SWANPORT TAS 7190"

# robust, case-insensitive match on FULL_ADDRESS
cand <- joined_data %>%
  mutate(FULL_ADDRESS_UP = str_squish(str_to_upper(FULL_ADDRESS))) %>%
  filter(FULL_ADDRESS_UP == str_squish(str_to_upper(target_addr)))

if (nrow(cand) == 0L) stop("Address not found in joined_data$FULL_ADDRESS. Check spelling/for")
# If multiple matches, take the first
ap <- cand %>% slice(1)

# Align CRS and extract raster value at the point
ap_rcrs <- st_transform(ap, st_crs(r))

# Convert to terra vector and extract
ap_vect <- vect(ap_rcrs)
val_df <- terra::extract(r, ap_vect, method = "simple")

# val_df has columns: ID (row index), and the layer name of r (e.g., "Barley")
col_val <- names(val_df)[2]

```

```

cell_code <- val_df[[col_val]][1]

# Map code -> label using VAT
label <- vat[[lab_col]][match(cell_code, vat[[val_col]])]

# Report (and quick sanity check) -----
message(sprintf("Address: %s", target_addr))

```

Address: 401 BRESNEHANS ROAD LITTLE SWANPORT TAS 7190

```

library(exactextractr)

# Load raster + VAT
r <- rast("ESM_Tiff/ESM_Tiff/Barley.tif")

vat <- read.dbf("ESM_Tiff/ESM_Tiff/Barley.tif.vat.dbf", as.is = TRUE)

# Bind raster attribute table (RAT) so codes map to labels
val_col <- grep("^value$", names(vat), ignore.case = TRUE, value = TRUE)
if (length(val_col) != 1L) stop("Could not identify VAT 'Value' column in DBF.")

lab_col <- grep("(suitab|class|label|cat)", names(vat), ignore.case = TRUE, value = TRUE)
if (length(lab_col) == 0L) {
  warning("No label-like column found in VAT; labels will equal the numeric code.")
  lab_col <- val_col
}

# Try to find RGB columns for colors
r_col <- grep("^r(ed)?$", names(vat), ignore.case = TRUE, value = TRUE)
g_col <- grep("^g(reen)?$", names(vat), ignore.case = TRUE, value = TRUE)
b_col <- grep("^b(lue)?$", names(vat), ignore.case = TRUE, value = TRUE)
has_rgb <- length(r_col) == 1L && length(g_col) == 1L && length(b_col) == 1L

# Build lookup and bind as RAT
rat <- data.frame(
  ID = vat[[val_col]],
  Category = as.character(vat[[lab_col]]),
  stringsAsFactors = FALSE
)
levels(r) <- rat

```

```

# Also keep a tidy lookup for joining later
vat_lu <- tibble(
  code = vat[[val_col]],
  label = as.character(vat[[lab_col]])
)
if (has_rgb) {
  to_hex <- function(r,g,b) sprintf("#%02X%02X%02X", pmin(255, as.integer(r)),
                                                    pmin(255, as.integer(g)), pmin(255, as.integer(b)))
  vat_lu$color <- to_hex(vat[[r_col]], vat[[g_col]], vat[[b_col]])
} else {
  vat_lu$color <- NA_character_
}

# Select the address polygon from `joined_data`
target_addr <- "401 BRESNEHANS ROAD LITTLE SWANPORT TAS 7190"

parcel <- joined_data %>%
  mutate(FULL_ADDRESS_UP = str_squish(str_to_upper(FULL_ADDRESS))) %>%
  filter(FULL_ADDRESS_UP == str_squish(str_to_upper(target_addr))) %>%
  slice(1) # take first if has multiple PIDs

if (nrow(parcel) == 0L) stop("Address not found in joined_data$FULL_ADDRESS.")

# Re-project polygon to raster CRS
parcel_r <- st_transform(parcel, st_crs(r))

# Extract cell-level values + coverage fractions over the polygon
# <layer_name> (the raster code) and coverage_fraction
cell_df <- exact_extract(r, parcel_r, fun = NULL, progress = FALSE)[[1]]

# Standardize the value column name to 'value'
val_name <- setdiff(names(cell_df), c("coverage_fraction"))
if (length(val_name) != 1L) stop("Unexpected columns from exact_extract().")
cell_df <- cell_df %>% rename(value = !!val_name)

# Majority rule (>= 50% covered) + tabulate by class
majority_tab <- cell_df %>%
  filter(!is.na(value), coverage_fraction >= 0.5) %>% # 50% threshold
  group_by(value) %>%
  summarise(pixels = n(), .groups = "drop") %>%
  left_join(vat_lu, by = c("value" = "code")) %>%
  arrange(desc(pixels)) %>%

```

```

  select(class_code = value, class_label = label, color, pixels)

# Identify VAT keys (already loaded: `vat`, `cell_df`)
val_col <- grep("^value$", names(vat), ignore.case = TRUE, value = TRUE)
if (length(val_col) != 1L) stop("Could not identify VAT 'Value' (code) column.")

# pick a human-readable label column (e.g., Suitabilit, CLASS, Label, Category)
lab_col <- grep("(suitab|class|label|cat)", names(vat), ignore.case = TRUE, value = TRUE)[1]
if (is.na(lab_col)) {
  warning("No label-like column found in VAT; falling back to numeric Value as label.")
  lab_col <- val_col
}

# Ensure the extraction's raster column is named 'value'
cell_df <- cell_df %>% rename(value = !!val_name)

# Join to VAT to map internal codes -> logical class
cell_df2 <- cell_df %>%
  dplyr::left_join(vat %>% dplyr::select(all_of(c(val_col, lab_col))),
    by = c("value" = val_col))

# Majority rule (>= 50% covered) & collapse by class label
majority_tab <- cell_df2 %>%
  dplyr::filter(!is.na(.data[[lab_col]]), coverage_fraction >= 0.5) %>%
  dplyr::group_by(class_label = .data[[lab_col]]) %>%
  dplyr::summarise(pixels = dplyr::n(), .groups = "drop") %>%
  dplyr::arrange(dplyr::desc(pixels))

# print
print(majority_tab, n = nrow(majority_tab), width = Inf)

```

```

# A tibble: 5 x 2
  class_label                pixels
  <chr>                  <int>
1 4.0 Unsuitable             254
2 2.0 Suitable              131
3 2.1 Suitable (with soil management)    17
4 1.1 Well suited (with soil management)    5
5 1.0 Well suited             4

```

```
cat("\nTotal pixels (majority-rule):", sum(majority_tab$pixels), "\n")
```

Total pixels (majority-rule): 411

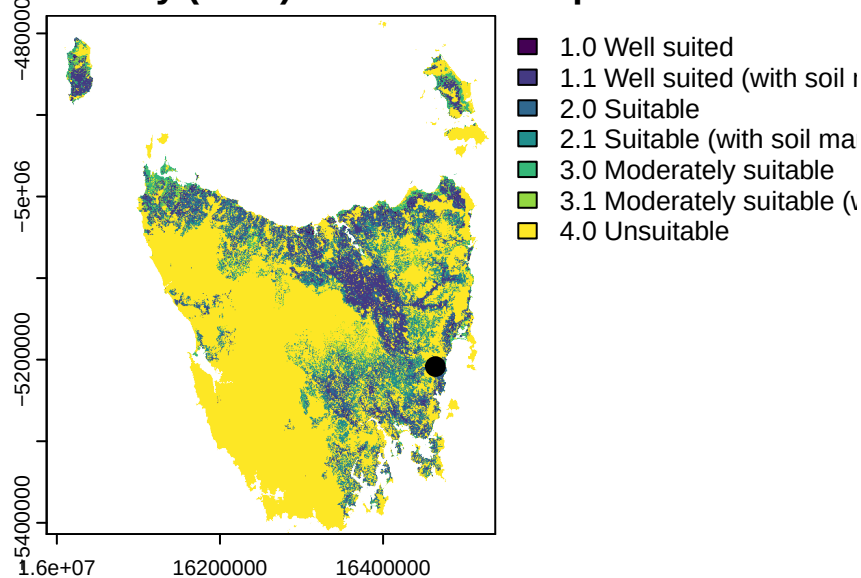
```
# ---- (Optional) Map with parcel outline + pinpoint -----
# representative point guaranteed to lie within polygon
pt_sf <- sf::st_point_on_surface(parcel_r)      # sf POINT, same CRS as raster

# Plot the categorical raster with legend (labels come from RAT binding)
plot(r, main = "Barley suitability (ESM) - with address point")

# Overlay parcel outline
plot(sf::st_geometry(parcel_r), add = TRUE, border = "black", lwd = 1.5)

# Overlay the pinpoint (black dot)
points(sf::st_coordinates(pt_sf), pch = 19, cex = 1.3)
```

Barley suitability (ESM) – with address point



```
# Caption in console (optional)
cat("\nPlotted:", target_addr, " (black dot marks representative point; outline = parcel)\n")
```

Plotted: 401 BRESNEHANS ROAD LITTLE SWANPORT TAS 7190 (black dot marks representative point)

```
# Compute bounding box of the parcel
bbox <- sf::st_bbox(parcel_r)

# Add a buffer of 50 units (meters if CRS is projected)
buffer <- 50
xlim <- c(bbox["xmin"] - buffer, bbox["xmax"] + buffer)
ylim <- c(bbox["ymin"] - buffer, bbox["ymax"] + buffer)

# Plot raster zoomed to bounding box + buffer
plot(r, main = "Barley suitability (zoomed on parcel)",
     xlim = xlim, ylim = ylim)

# Overlay parcel outline
plot(sf::st_geometry(parcel_r),
     add = TRUE, border = "black", lwd = 2)

# Fill polygon semi-transparent (optional)
plot(sf::st_geometry(parcel_r),
     add = TRUE, col = adjustcolor("black", alpha.f = 0.2), border = NA)

# Add representative point
pt_sf <- sf::st_point_on_surface(parcel_r)
```

Warning: st_point_on_surface assumes attributes are constant over geometries

```
points(sf::st_coordinates(pt_sf), pch = 19, col = "black", cex = 1.3)
```


Barley suitability (zoomed on parcel)

